

Mixture-of-Experts Action Chunking Transformers for High-Precision Robot Imitation Learning

Hung Phan Quoc Mai^{1,2} Naeem Ul Islam²

¹National Economics University, Vietnam ²Yuan Ze University, Taiwan

Correspondence: naeem@saturn.yzu.edu.tw

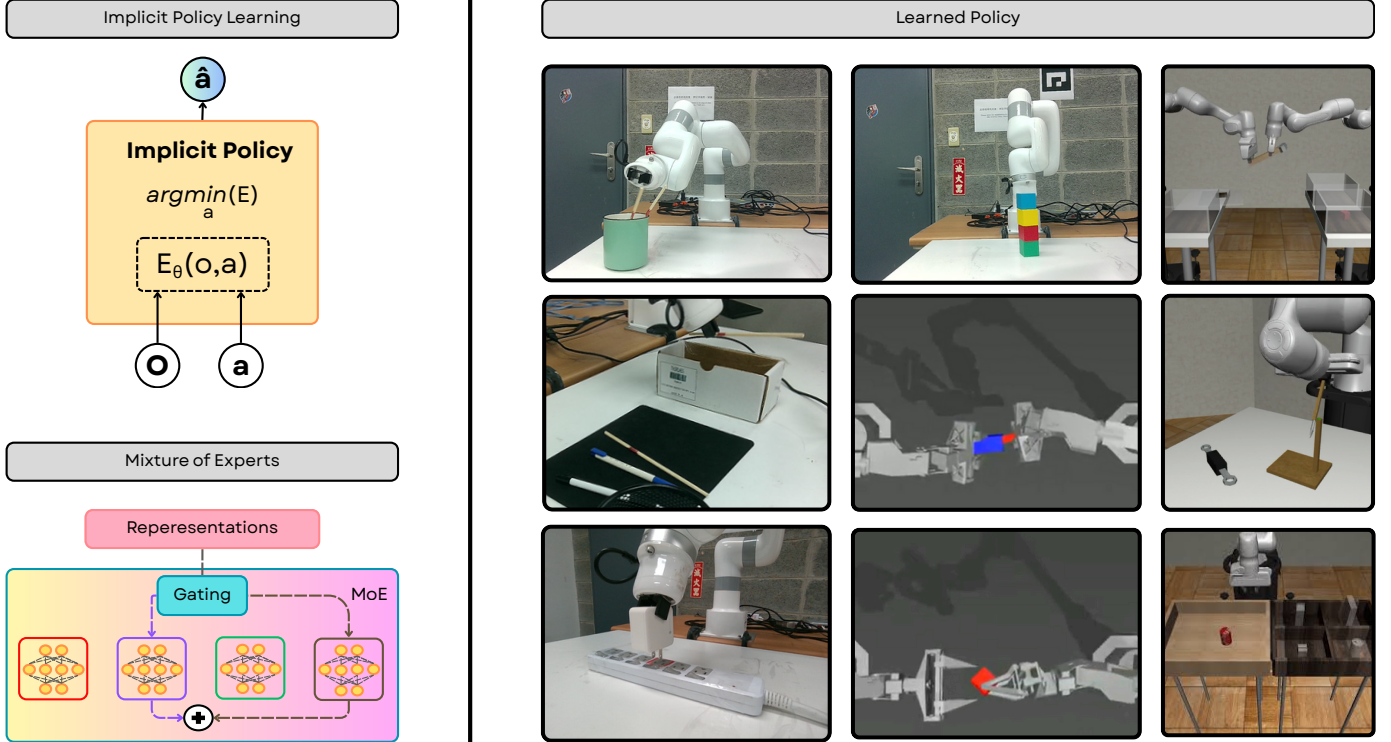


Fig. 1: **MEAT: Mixture of Experts in Action Transformer Policy.** *Upper Left:* Implicit policy learns a function conditioned on both action and observation and optimizes for actions that minimize the error. *Bottom Left:* The visualization of Mixture of Experts routing in our new model, a part of the Transformer Policy. *Right:* Examples of learned policy and deployed to the real robot. We will showcase of both simulation and real robots.

Abstract—Learning robot control policies from demonstrations is typically framed as a supervised behavior cloning problem, where policies predict actions based on complex sensory inputs. Despite its simplicity, this approach faces challenges like multi-modal action distributions, long-term dependencies, and the need for precision in manipulation tasks.

Recent advancements in imitation learning have introduced generative and sequence modeling architectures, such as Variational Autoencoders (VAEs), diffusion models, and Transformers. These models excel at capturing action diversity and integrating modalities such as vision and language, which enhance the generalization of robot behavior.

However, precision-critical tasks still struggle with error accumulation from covariate shifts and prediction errors. Action chunking policies address this by jointly predicting future action sequences, improving temporal consistency, and reducing noise. Additionally, Mixture-of-Experts (MoE) architectures effectively scale the capacity of Transformers and enable expert specialization, as seen in models like DeepSeek and Kimi.

This work integrates a MoE mechanism into the Action

Chunking Transformer (ACT) framework, exploring the impact of expert balancing and regularization on performance. Evaluations in simulated and real-world settings show accuracy improvements of 5–30% over standard ACT baselines, indicating that MoE-enhanced Transformers are a promising solution for high-precision robot control from demonstrations.

Index Terms—Imitation Learning, Human teaching Robot, Robot Control Policy

Note to Practitioners

This paper introduces a Mixture of Experts (MoE)-enhanced Action Chunking Transformer (ACT), which predicts multiple future actions jointly while leveraging specialized expert modules for different contexts. For practitioners, this means a more robust and adaptable policy that can reduce noise in execution and improve accuracy in both simulation and real-world deployments. The approach is especially relevant for applications that require long-horizon decision-making, such as collaborative robots in manufacturing, autonomous

warehouse systems, or precision surgical tools. Although the current evaluation demonstrates significant accuracy improvements, further work is needed to validate performance across diverse robot platforms and integrate the method into end-to-end training pipelines for real-time deployment. This work presents a Mixture of Experts (MoE)-enhanced Action Chunking Transformer (ACT) that jointly predicts multiple future actions by utilizing specialized expert modules for different contexts. This provides a more resilient and flexible strategy that can increase accuracy in simulation and real-world deployments while reducing execution noise to meet the demands of practitioners. This methodology is particularly applicable to applications that require long-term decision-making, such as autonomous warehousing systems, precision surgical instruments, or collaborative robots in manufacturing.

I. INTRODUCTION

At its core, learning robotic manipulation policies from demonstrations can be framed as a supervised regression problem, where the goal is to map observations to actions, which is so-called imitation learning. Imitation learning is a powerful paradigm to train agents from expert demonstrations to learn versatile skills [24, 27, 29]. However, predicting robot actions presents unique challenges that set it apart from typical supervised learning tasks. These include handling multimodal action distributions, maintaining temporal consistency across sequences, and meeting the demands of high-precision control. Recent research has shown promise in building autonomous robot control models by learning from human demonstrations via imitation learning, often leveraging advanced architectures like Transformers [43], Diffusion models [5, 35], or VAEs [31] for policy learning. Despite learning extensively from human behavior, these models still face significant limitations.

Transformer-based approaches are particularly notable for their ability to integrate natural language instructions with rich and flexible vector representations, which has led to their widespread adoption in recent robot control models such as the Action Chunking Transformer (ACT) [43], Octo [13], and π_0 [2]. These models leverage the transformer architecture’s capacity for modeling long-range dependencies and multimodal inputs to map observations and instructions to action sequences effectively. Nonetheless, tasks that require high precision and rely heavily on visual feedback remain especially challenging for imitation learning, even when trained on high-quality demonstrations. Small prediction errors in actions can accumulate over time, causing significant deviations in state and exacerbating the well-known compounding error problem associated with imitation learning [43]. To mitigate such issues, the ACT policy predicts joint positions for the next k timesteps jointly as an *action chunk*, rather than predicting one step at a time. This reduces the effective task horizon and helps the model handle temporal confounders such as pauses or noise in the demonstration trajectories more robustly.

However, despite these advances, existing transformer-based policies remain relatively simple in their architectural design, leaving considerable room for further improvement. Originally

designed for natural language processing, transformers have since seen numerous architectural enhancements, one of the most prominent being the Mixture of Experts (MoE) [17] technique. MoE introduces sparse expert routing to improve scalability and expressiveness, and has shown strong effectiveness in cutting-edge models such as DeepSeek [7] and Mixtral [17]. These successes underscore the potential of MoE and related innovations to further enhance transformer-based policies for robot control and imitation learning tasks.

In this study, we explore model-level enhancements to the Action Chunking Transformer (ACT) by incorporating MoE into its architecture and investigating the effectiveness of auxiliary loss functions during training. Our experiments demonstrate that augmenting ACT with MoE improves task performance and robustness, offering a promising direction for scaling up to larger, multimodal robot control models (**Figure 1**). We validate our proposed approach through extensive simulations on benchmark datasets.

In general, our main contributions can be summarized as follows:

- We represent MEAT (Mixture of Experts in Action Transformer policy), where we propose an enhanced version of the ACT model by integrating a MoE mechanism into the transformer layers, enabling better utilization of model capacity and improving generalization.
- We investigate the impact of auxiliary loss functions on training stability and performance of the ACT model, providing insights into effective training strategies for action-chunking policies.
- We empirically demonstrate that MEAT achieves 5-15% higher accuracy compared to the original ACT across multiple simulated experiments.
- We evaluate and deploy our MEAT model on both the dataset proposed in the original ALOHA paper as well as the widely-used RoboMimic [23] dataset, showcasing its effectiveness and adaptability across different benchmarks.

II. RELATED WORKS

Advances in Imitation Learning. Many of recent research has been directed towards imitation learning for robots, and many approaches have been presented. Since the success of the Diffusion model [16] and the Transformer model [39], these models are widely studied and applied for use as a policy for robotics manipulation. [5] introduces Diffusion Policy, a conditional denoising diffusion approach for robot visuomotor control that enables stable training, handles multimodal actions, and supports high-dimensional spaces using receding horizon control, visual conditioning, and a time-series diffusion transformer. DiT-Block Policy [6] improved diffusion transformer architecture with adaLN-zero layers and optimized input encodings, achieving state-of-the-art performance on long-horizon bimanual dexterous tasks and demonstrating strong scaling on 10 hours of multimodal, language-annotated ALOHA demonstrations. In addition, the approach

using transformer-based models is the most studied, which will be discussed in more detail below.

Transformer models in Robotics Transformer-based models offer the advantage of unifying diverse tasks such as vision, language commands, and speech into a single end-to-end framework using vector embeddings, enabling the creation of foundation models that can be further fine-tuned. [43] implement action chunking policies using Transformers, training them as conditional VAEs (CVAEs) to capture human data variability. Action Chunking with Transformers (ACT) significantly outperforms previous imitation learning approaches on various simulated and real-world fine manipulation tasks. [13] presents Octo, a large transformer-based policy trained on 800k trajectories from the Open X-Embodiment dataset, designed as a generalist foundation model for robotic manipulation that can be efficiently finetuned across diverse platforms, sensors, and action spaces using language or image goals. With a similar approach, π_0 [2] proposes a generalist robot policy using a novel flow matching architecture built on a pre-trained vision-language model to leverage semantic knowledge for dexterous tasks, demonstrating strong performance across diverse platforms and tasks through prompting, instruction following, and fine-tuning. Rather than using Transformer for learning manipulating policy, Q-Transformer [32] introduces a scalable offline reinforcement learning method that leverages both human and autonomous data by modeling Q-functions with a Transformer. By discretizing action dimensions into tokens, the approach enables effective sequence modeling for Q-learning and demonstrates strong performance across diverse real-world robotic manipulation tasks.

Mixture of Experts. MoE models selectively route information through a network, using a gating mechanism to activate only a subset of specialized expert modules per input. This approach was popularized by [38], which introduced conditional routing via a gating network. With the success of Transformers, MoE architectures evolved most notably with Switch Transformers [9], which incorporated sparse expert feed-forward layers to scale model capacity while keeping computation efficient. Sparsely activated expert networks within layers allowing only a subset of experts to process each input, which improves model capacity and efficiency. This approach enables scaling to larger models while maintaining manageable computational costs. This design became foundational for many Large Language Models: Mixtral [17], Deepseek-v3 [7], GLaM [8], and gpt-oss [40]. Notably, at the time Mixtral 8x7B – Instruct [17] achieving superior performance over GPT-3.5 Turbo, Claude-2.1, Gemini Pro, and Llama 2 70B on human evaluation benchmarks. In addition, the components of MoE are also studied including: Gating Function, Expert Network, Routing Strategy, Training Strategy. Jointly optimizing both routers and experts poses challenges like expert collapse, where multiple experts converge to similar functions, and router collapse, where only a few experts are consistently selected. Load balancing losses [9, 38] address this by encouraging more uniform expert usage. Various routing strategies have since been proposed, including expert choice

routing [44], differential k-selection [14], frozen hashing [36], and linear assignment [20]. Instead of learning an auxiliary loss for gating expert balancing, [41] introduced Loss-Free Balancing, a gradient-free load balancing method for MoE models that avoids auxiliary loss interference by applying expert-wise biases to routing scores, dynamically adjusting them based on recent usage to maintain balanced expert load. In robotics arm manipulation policy learning, the idea of using MoE is gradually being studied more. One such approach is MoDE [34] which introduces Mixture-of-Denoising Experts in Diffusion Transformer Policy that surpasses current state-of-the-art Diffusion Policies while enabling parameter-efficient scaling through sparse experts and noise-conditioned routing. MoVEInt [31] proposes a Mixture-of-Experts VAE framework that learns a shared latent space from human demonstrations using an MDN prior, enabling accurate and reactive robot actions in human-robot interactions.

Demonstration Datasets. In order to facilitate the fast-growing Imitation Learning, many datasets are introduced from real environments to simulated environments. For example, in real-world environments, Open X-embodiment [26] introduces a large, standardized dataset of 160,266 tasks across 22 robots from 21 institutions to enable training and evaluation of generalist robot policies adaptable across diverse platforms and skills. Besides, there are numerous of demonstration datasets have been published such as DROID [18] with 76k demonstration trajectories or 350h of interaction data, collected across 564 scenes and 86 tasks by 50 data collectors; and Mobile-ALOHA [11] which is a low-cost and whole-body teleoperation system for data collection. In simulation, Robomimic [23] presents an extensive evaluation of six offline learning algorithms on diverse simulated and real-world multi-stage manipulation tasks using human demonstration datasets, highlighting key challenges like algorithm sensitivity, demonstration quality, and evaluation variability, while emphasizing the potential to learn complex policies from raw sensory data; all datasets and code are open-sourced to support future research. Further developed on Robomimic, MimicGen [22] is a system that automatically synthesizes large-scale, diverse robot demonstration datasets from a small set of human examples by adapting them to new contexts, enabling effective imitation learning for complex, long-horizon tasks and offering a cost-efficient alternative to collecting extensive human demonstrations.

III. APPROACH

A. Problem Formulation

We aim to learn a policy that predicts chunks of consecutive actions based on the current observation and a latent variable inferred from the demonstration. Given a dataset of trajectories $\mathcal{D}$, at each timestep the policy outputs a sequence of k future actions conditioned on the observation o_t and a latent embedding z . The latent z is obtained by encoding the ground-truth action chunk and observation history, while the policy decodes z and o_t to reconstruct the action chunk. The training objective combines a reconstruction loss that minimizes the

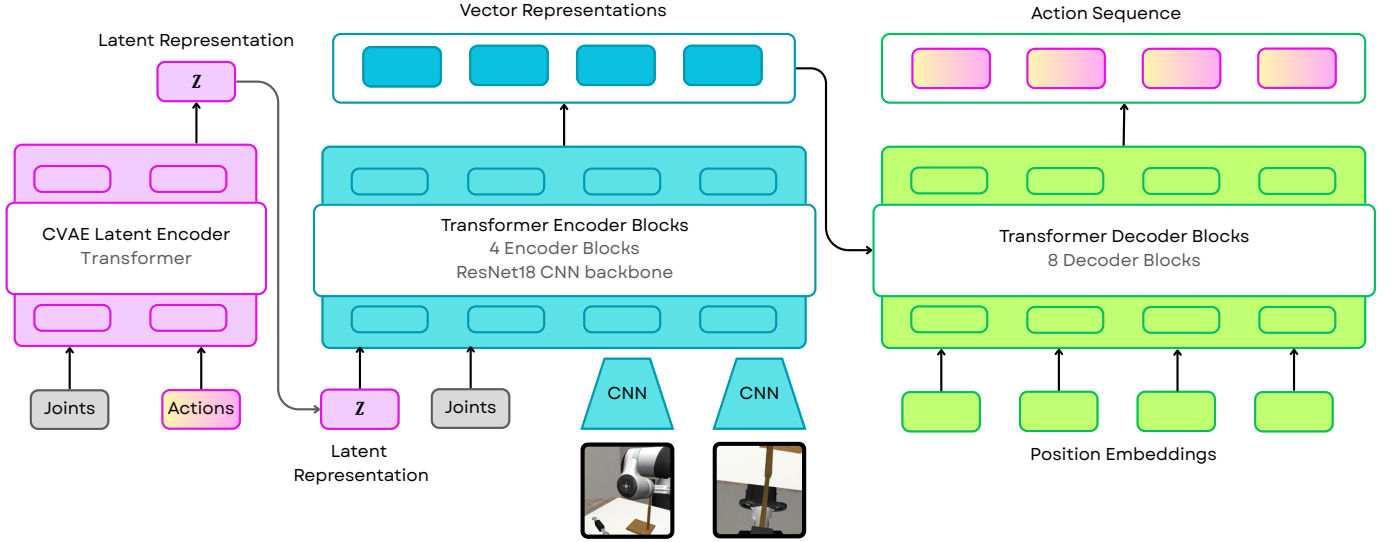


Fig. 2: **Architecture of the overall MEAT (ACT) model.** MEAT (ACT) is trained under a Conditional VAE (CVAE) framework with an encoder-decoder architecture. The encoder maps the action sequence and joint observations into a latent style variable z , which is discarded during inference. The decoder (policy) employs a transformer encoder to synthesize images from multiple viewpoints, joint positions, and z , and a transformer decoder to predict the corresponding sequence of actions. At test time, z is set to the mean of the prior distribution (i.e., zero).

mean squared error between the predicted and demonstrated action chunks:

$$\mathcal{L}_{\text{reconst}} = \mathbb{E}_{\mathcal{D}} [\text{MSE}(\hat{a}_{t:t+k}, a_{t:t+k})] \quad (1)$$

and a regularization loss that encourages the latent variable to follow a standard Gaussian prior:

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_{\mathcal{D}} [D_{\text{KL}}(q_{\phi}(z|a_{t:t+k}, \bar{o}_t) \| \mathcal{N}(0, I))]. \quad (2)$$

B. Action Chunking Transformer Policy

To mitigate the compounding error problem inherent in imitation learning, particularly for high-frequency, long-horizon tasks, ACT adopt the concept of *action chunking*, inspired by findings in neuroscience [19], where sequences of actions are grouped and executed as a single unit. This approach reduces the effective task horizon by a factor of k , which improves robustness and allows the policy to handle non-Markovian behaviors observed in human demonstrations, such as pauses or temporally correlated noise [43].

Specifically, rather than predicting a single action a_t at each timestep given the current observation s_t , the ACT policy predicts a chunk of k future actions jointly:

$$\pi_{\theta}(a_{t:t+k-1} | s_t), \quad (3)$$

where $a_{t:t+k-1} = (a_t, a_{t+1}, \dots, a_{t+k-1})$ denotes the predicted chunk. At inference time, every k steps, the agent observes s_t , samples z from the encoder, and generates the next k actions, which are then executed sequentially.

However, simply updating the observation every k steps can result in abrupt and jerky motion. To alleviate this, ACT employs a *temporal ensembling* strategy. Instead of discarding intermediate observations, the policy is queried at every timestep, producing overlapping predictions for the same

future timestep from multiple chunks. These predictions are then aggregated through a weighted average:

$$\hat{a}_t = \frac{\sum_{i=0}^{k-1} w_i A_i[t]}{\sum_{i=0}^{k-1} w_i}, \quad (4)$$

where $A_i[t]$ is the i -th prediction of the action at timestep t from the overlapping chunks, and w_i is an exponentially decaying weight given by:

$$w_i = \exp(-m \cdot i), \quad (5)$$

where m controls how quickly newer observations are incorporated. Smaller m allows faster adaptation to new observations.

This temporal ensembling ensures that action predictions are smooth and continuously updated, while maintaining the benefits of reduced horizon and robustness to temporal confounders. Importantly, this ensemble operates only at inference time and does not introduce additional training cost.

Figure 2 illustrates the architecture of a Transformer policy used for visuomotor robotic manipulation. The model follows an encoder-decoder Transformer structure originated from the idea of [4], where multimodal sensory inputs, which include images processed by a ResNet18 CNN, joint positions, and a latent vector, are encoded into a vector representation using a stack of 4 Transformer encoder blocks. The latent vector representation produced by CVAE Latent Encoder, and only be used during training, for inference time it is set to $\mathbf{0}$. These encoded features are then passed to 8 Transformer decoder blocks to autoregressively generate the action sequence, conditioned on learned position embeddings.

C. Mixture of Experts

To evaluate the efficiency of advanced Transformer architectures, which are widely adopted in NLP for robotic

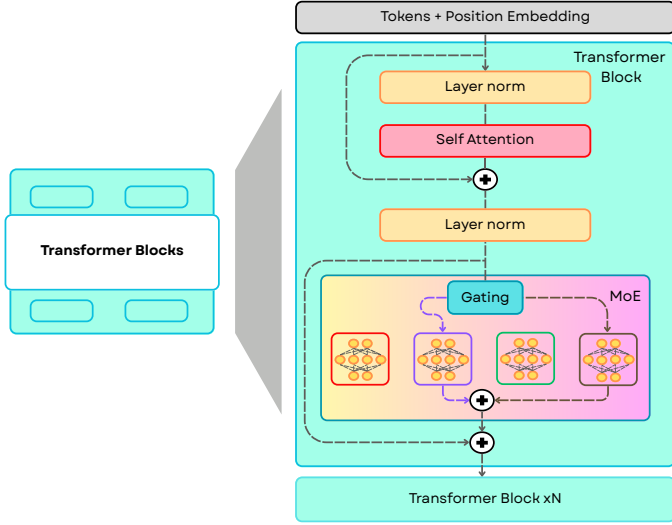


Fig. 3: **Transformer Block architecture composition including Mixture of Experts.** Following processing through the Self-Attention mechanism and Layer Normalization, the embedding vectors are subsequently input into a gating function, which adaptively selects two experts from a pool of four for each token.

arm manipulation, we introduce a Mixture-of-Experts (MoE) design into the Transformer policy. The key innovation lies in the incorporation of MoE within the Transformer blocks (as detailed on the right side of the figure). Each Transformer block contains a MoE sub-layer consisting of multiple expert networks and a gating mechanism that dynamically selects the top-k experts per input token. This design increases model capacity without proportionally increasing computation, allowing more expressive policies while maintaining efficiency. This MoE-enhanced Transformer policy significantly boosts the capability of robotic arms in complex tasks by enabling specialization across experts for different manipulation sub-tasks or observation types.

Figure 3 describes the Transformer block augmented with a Mixture-of-Experts (MoE) layer. After the standard self-attention and normalization layers, tokens are passed through a gating mechanism that routes them to a sparse subset of expert networks within the MoE module. Each expert is a small feedforward network, and only a few are activated per input, improving model capacity without increasing computational cost proportionally. In our experiment, we set 4 experts in each Transformer block, of which 2 experts are activated.

To preserve gradient flow through the non-differentiable sampling step, we scale each expert’s output by its routing probability and renormalize the selected probabilities. To mitigate expert collapse, we add a load balancing loss that encourages more uniform expert utilization [9]:

$$LB(\sigma_t) = N \sum_{n=1}^N \frac{1}{|B|} \left(\sum_{i=1}^{|B|} \mathbb{1}(\phi(\sigma_t)_i)_n > 0 \right) \times \frac{1}{|B|} \left(\sum_{i=1}^{|B|} i = \mathbb{1}^{|B|} \text{softmax}(\phi(\sigma_t)W_R)_n \right) \quad (6)$$

D. Auxiliary-Loss-Free Load Balancing Strategy

In addition to employing the standard MoE architecture, we also evaluate the Auxiliary-Loss-Free Balancing Strategy proposed in [41]. Specifically, instead of relying solely on naive top-K gating, [41] introduce an expert-specific bias term $\{b_i\}_{i=1}^N$ to the gating scores $s_{i,t}$. These biased scores, $s_{i,t} + b_i$, are used to determine the top-K experts for routing, thereby promoting a more balanced expert utilization:

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{TopK}(\{s_{j,t} + b_j \mid 1 \leq j \leq N\}, K) \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

It is important to note that the expert bias term b_i is used solely to influence the top-K routing decision. It does not affect the final weighted combination of expert outputs—i.e., $g_{i,t}$ is not modified by b_i during the actual MoE output computation.

IV. SIMULATION EXPERIMENTS

A. Dataset

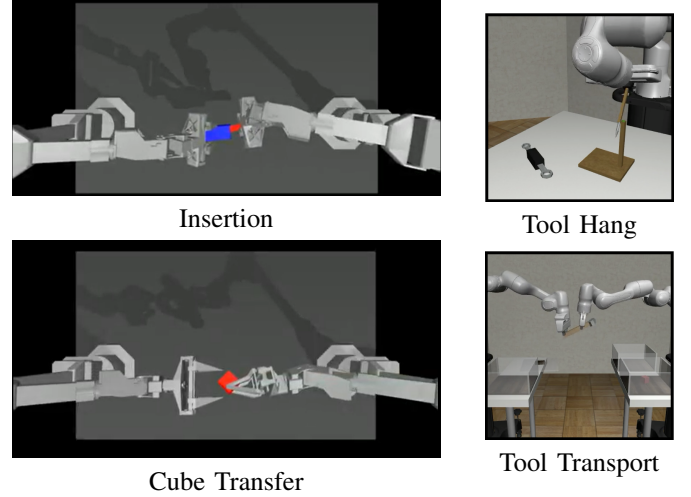


Fig. 4: **Illustrations of 4 Simulated tasks.** Insertion and Cube Transfer from ALOHA (Left), Tool Hang and Tool Transport from RoboMimic (Right).

For our experiments, we evaluate on 4 simulation tasks derived from publicly available datasets: 2 simulated tasks Transfer Cube and Bimanual Insertion introduced by ALOHA [43], and 2 RoboMimic tasks Tool Hang and Tool Transfer proposed by [23] as shown in **Figure 4**. Among Robomimic tasks, we chose 2 tasks, Tool Hang and Tool Transfer, as they are the most challenging since they require many steps and

dexterous to complete. The simulated dataset from ALOHA contains two types of demonstrations: scripted and human demonstrations, whereas RoboMimic provides only demonstrations performed by proficient humans. Human demonstrations are often noisier, less consistent, and harder for the robot to learn from compared to scripted ones. This is because humans may make small mistakes and immediately correct them, which behavior that can confuse the robot during imitation learning.

In the **Transfer Cube** task, the right robotic arm must first grasp a red cube from the table and accurately place it into the gripper of the left arm. This task is highly sensitive to precision, as the clearance between the cube and the left gripper is approximately 1 cm; even small errors can lead to collisions or task failure. In the **Bimanual Insertion** task, both arms are required to collaborate: one grasping a socket and the other a peg, and perform a mid-air insertion such that the peg aligns with the pins inside the socket. This task demands even higher precision, with a clearance of roughly 5 mm during the insertion phase.

For RoboMimic tasks, **Transport Tool** involves two robotic arms coordinating to move a hammer from a closed container to a target bin on a shelf. One arm retrieves the hammer, while the other clears trash from the target bin before completing a mid-air handover. And **Tool Hang** requires a robot to assemble a hook-and-base frame and hang a wrench on the hook, which demanding precise, multi-stage, rotation-intensive manipulation, making it the most challenging task.

B. Experiment Setup

We conduct all training and inference on an RTX 4080 GPU with 16GB of memory. Each task is trained using 3 random seeds. When training ACT models, for ALOHA tasks, training takes approximately 40 minutes per run for 3000 epochs. In contrast, RoboMimic tasks require around 1.5 hours per run with 4000 epochs due to their higher complexity and larger dataset size. We trained ACT with only one camera view: top view for ALOHA tasks and agent view for RoboMimic tasks, both using an image size of 480x640. We use full of 50 demos of ALOHA tasks, but only 150 of 200 demos of RoboMimic tasks due to resource restrictions. The ACT model has 84M parameters, while the MoE model contains 281M parameters, of which only 189M are activated per token during inference.

C. Evaluation Method

We present the best-performing method for each baseline on each benchmark, sourced from all possible sources: our reproduced results (ACT, BC-RNN, Diffusion Policy); or the results reported in the original papers (BC-ConvMLP, BeT, RT-1, VINN, IBC) since these models are reported as inefficient, so there is no need to reproduce. For BC-RNN, to align with the training setup used in ACT and the original Robomimic paper, we train the model using only one camera view (agent view), and increase the batch size as well as the number of training epochs from 600 to 1000. Evaluation is performed over 50 rollouts per seed among 3 seeds, resulting

in roughly 3000 total evaluation rollouts after 18 training runs per model architecture.

1) *ALOHA tasks*: We compare ACT models against four prior imitation learning methods. BC-ConvMLP is a simple yet widely adopted baseline with vanilla Convolution and MLP network, which processes current image observations using a convolutional network, concatenates its output with joint positions, and predicts the action. BeT [37] also employs a Transformer architecture but differs from ACT in two key ways: (1) it does not use action chunking, instead predicting a single action based on the observation history; and (2) it uses a separately trained, frozen visual encoder for image features, meaning perception and control are not jointly optimized. RT-1 [3] is another Transformer-based model that predicts one action from a fixed-length observation history. Both BeT and RT-1 discretize the action space, outputting a categorical distribution over discrete bins, with BeT adding a continuous offset from the bin center. In contrast, ACT directly predicts continuous actions, which is better suited for precise manipulation tasks. Finally, VINN [28] is a non-parametric method that requires access to the demonstration data at test time: it retrieves the k most visually similar observations from the demonstrations and predicts the action via weighted k -nearest-neighbors.

2) *RoboMimic tasks*: We compare RoboMimic data result with 2 models as benchmarks. Implicit Behavior Cloning (IBC) [10] leverages implicit energy-based models to learn policies that better capture complex, multimodal, and discontinuous behaviors from demonstrations, often outperforming explicit methods and achieving competitive or superior performance even on challenging, high-dimensional robot learning tasks without using rewards. BC-RNN [23], a recurrent variant of BC that models temporal dependencies in demonstrations via an RNN policy, is the best-performing method reported in the RoboMimic paper, achieving strong results by effectively leveraging sequential structure in the data.

D. Results

The experiment result of tasks from ALOHA dataset is represented in **Table I**. Unfortunately, we can not reproduce exactly the reported result in ALOHA paper [43], however some tasks achieve higher results (insertion scripted, transfer cube human) while the other achieve lower results (insertion human, transfer cube scripted). From the result table, it is clear that MoE-ACT consistently outperforms all other models, achieving the highest scores in most categories. In particular, in the Insertion human task, MoE-ACT achieves 85% in grasp, 60% in contact, and 21% in insert, significantly surpassing ACT and other baselines. Similarly, in Insertion scripted, MoE-ACT reaches near-perfect scores (97% grasp, 88% contact, 97% insert). On the Transfer cube tasks, MoE-ACT again leads with 65–97% success in the human setting and 99–93–85% in the scripted setting. Other models, such as ACT and MoE-ACT Aux, also perform competitively but generally lower than MoE-ACT. Notably, baseline models like BC-ConvMLP, BeT, RT-1, and VINN exhibit much lower

TABLE I: Success rate (%) for 2 simulated tasks from ALOHA paper. The bold Insert and Transfer means completion rate. MEAT outperforms the original ACT and other models.

Model	Insertion human			Insertion Scripted			Transfer cube human			Transfer cube scripted		
	Grasp	Contact	Insert	Grasp	Contact	Insert	Touch	Lift	Transfer	Touch	Lift	Transfer
BC-ConvMLP	0	0	0	5	1	1	3	1	0	34	17	1
BeT [37]	0	0	0	21	4	3	16	13	1	60	51	27
RT-1 [3]	0	0	0	2	0	1	4	2	0	44	33	2
VINN [28]	0	0	0	6	1	1	17	11	0	13	9	3
ACT [43]	73	55	11	96	88	33	97	55	55	97	79	80
MEAT	85	60	21	97	88	65	89	57	64	99	93	85
MEAT Aux-free	81	61	16	99	81	37	97	62	61	100	78	77

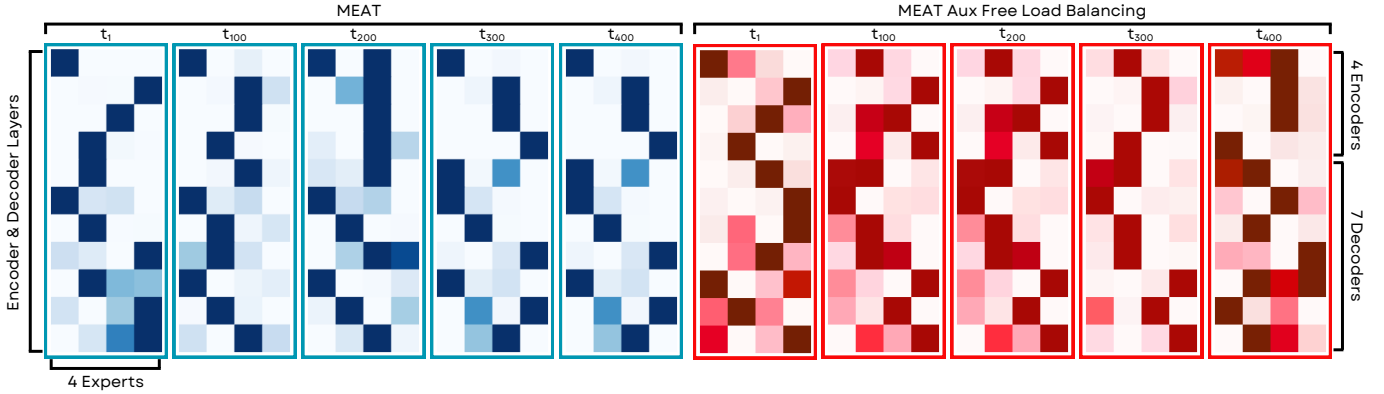


Fig. 5: **Visualized Expert Gating Samples.** The average usage of all experts in MEAT across all layers in some sampled timestep. Brighter blue/red corresponds to low usage and bolder to high one, and each image is separately normalized. Each matrix corresponds to 1 timestep during inference and has a size of 4x11 (which is 4 experts within 11 decoder and encoder layers). The activation shows that the Aux-Free load balancing strategy produces more noisy gating probabilities.

success rates, often close to 0% on several sub-tasks. These results demonstrate that MoE-ACT significantly improves task performance, particularly in challenging manipulation scenarios, and outperforms both the original ACT and all other compared methods.

TABLE II: Success rate (Highest | Average) for 2 simulated tasks from RoboMimic Proficient Human (ph) dataset. MEAT outperforms the original ACT and having competitive results with Diffusion Policy.

Model	Tool hang, ph	Tool transport, ph
IBC [10]	0 0	0 0
BC-RNN [23]	24 18	38 28
DP [5]	40 34	50 40
ACT [43]	46 38	44 34
MEAT	52 46	44 38
MEAT Aux-free	48 28	24 12

Table II reports the highest and average success rates on the Tool Hang and Tool Transport tasks from the RoboMimic dataset. For reproducing BC-RNN, as mentioned, we use only 1 camera view and increase the number of training epochs to align with the experimental setup in ACT. However, the

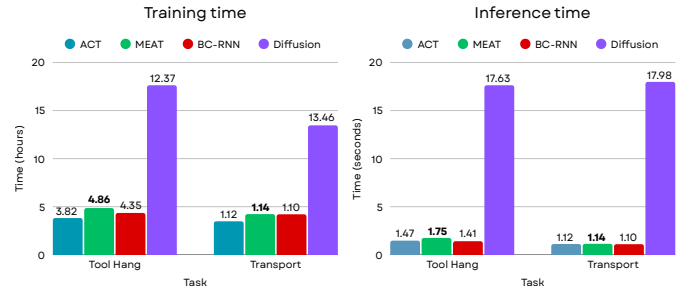


Fig. 6: **Training and Inference Time Benchmark.** Inference time is measured as the total time the model takes to predict an action, excluding the time taken to execute the action step. Training time is reported in hours, inference time is reported in seconds.

performance is significantly lower under this configuration. Our proposed MEAT consistently outperforms the original ACT in both tasks and has a competitive result compared with Diffusion Policy. On Tool Hang, MEAT achieves the highest | average success rate of 52 | 46, compared to ACT’s 46 | 38, showing improvements in both peak and overall performance. On the Tool Transport task, MEAT did not outperform DP in terms of overall accuracy. However, it still achieves a competitive average performance compared to DP. Importantly,

MEAT’s strengths lie in its efficient inference and training times, as well as its lightweight architecture. As shown in **Figure 6**, MEAT’s inference time is comparable to that of lighter models such as CNNMLP and ACT, and is approximately 16 times faster than that of DP. This is a significant advantage, considering that DP is known for its slow inference [21, 30], primarily due to its time-consuming denoising process. In our real-world experiments, we demonstrate that MEAT can be efficiently deployed on a laptop GPU, delivering real-time performance while maintaining competitive predictive accuracy.

E. Analysis

Although BC-RNN can yield good performance, we found that training it is often slow and requires a large amount of Random Access Memory (RAM). Specifically, we had to reduce the number of demonstrations from 200 to 150 in order to fit the entire dataset into 120GB of RAM and run training successfully. This is why, as mentioned, we used only 150 demonstrations for the RoboMimic tasks. Furthermore, BC-RNN takes significantly longer to train completely, approximately 6 hours for Tool Transport and 10 hours for Tool Hang, which, compared to ACT completes training in only 1.4 hours while still achieving better results. This further highlights the efficiency and effectiveness of MoE-ACT.

Regarding **Figure 6**, MEAT, ACT, and BC-RNN show no significant differences in inference time. In some cases, MEAT even infers faster than ACT or BC-RNN, particularly in the tool transfer task. Notably, the Diffusion Policy model exhibits extremely high inference and training times, which is up to 16 times longer in inference and 3 times longer in training compared to other models. While Diffusion Policy may offer slightly better accuracy, the gain is not substantial enough to justify its high computational cost, making it unclear whether it truly outperforms MEAT overall.

To better understand why MEAT with aux-free load balancing performs poorly, we analyze the expert utilization by visualizing the gating probabilities for each expert, as shown in **Figure 5**. The results clearly demonstrate that the standard MEAT model exhibits more focused and consistent expert selection, leading to cleaner and more concentrated gating decisions. In contrast, the aux-free load balancing variant displays significantly noisier and more scattered gating behavior, suggesting that the model has difficulty in confidently selecting which experts to activate. This noisy expert selection results in less specialization among the experts, meaning that each expert is not sufficiently or consistently trained. Consequently, the overall model performance suffers due to the lack of clear routing and expert specialization, which are critical for the effectiveness of MoE architectures. These observations indicate that MEAT with aux-free load balancing is currently ineffective, and further investigation is necessary to improve its expert routing mechanism, either by designing better load balancing strategies or by introducing alternative forms of regularization to promote expert diversity and specialization.

V. REALWORLD EXPERIMENTS

A. Setup

Teleoperation. To record human demonstration data for training models, we used two robotic arms: one as the leader and the other as the follower. Both are UFactory Lite 6 models, each with 6 degrees of freedom (DoF). During teleoperation, the human operator manually moves the leader robot arm. The system then transmits the current state of the leader to the follower, allowing the follower to replicate the leader’s movements in real time.

Recording data. As the follower robot executes the task, we simultaneously record the leader’s state (used as the action), and capture the follower’s joint positions, joint angles, and images of the scene involving the robot and the objects. Teleoperation and data recording are handled in parallel using two separate threads. Due to hardware limitations, we record robot states at a frequency of 20 Hz. For each task in real-world experiments, the robot’s state is represented as a 6-dimensional vector. We extend both the action and observation vectors to 7 dimensions to include the state of the end-effector. The end-effectors used in experiments are grippers and vacuums, which can operate in two basic states: open or closed.

Policy Training. Similar to the simulation tasks, we trained the policy on a single NVIDIA RTX 4080 GPU using data collected from the real robot. Training Transformer-based models on real-robot data typically requires more time, with an estimated 12,000 to 30,000 steps needed to achieve stable and reliable performance in real-world scenarios. During inference, the trained policy was deployed on a laptop equipped with an NVIDIA RTX 4060 GPU. The model takes real-time inputs, including RGB images and robot states, and outputs the corresponding actions, specifically, the predicted next-step position states for the robot.

B. Task 1: ‘Chopsticks’

Our first experiment is named **Chopsticks**. In this task, the robot is equipped with a gripper as its end-effector. As shown in **Figure 7**, the evaluation setup is as follows: two chopsticks are placed randomly within a designated box area on the table, and a cup is positioned beside them. The robot’s objective is to pick up each chopstick individually and place it into the cup. We trained ACT and MEAT models for 15,000 steps, which took approximately 4.5 hours on 50 demonstrations. Each raw trajectory record of Chopsticks tasks includes 390 to 430 timesteps; after data cleaning we refine all to fit into 400 timesteps.

Result Analysis. **Table III** reports the 50 experiments result for chopsticks and comparison among 3 policies, in the table *locate* means the gripper is correctly positioned over a chopstick, *lift* is that the chopstick is successfully grasped, and *put* is the chopstick is placed into the cup. The robot must complete all three stages for each of the 2 chopsticks. our model MEAT significantly outperforms both ACT and BC-ConvMLP across all stages of the Chopsticks

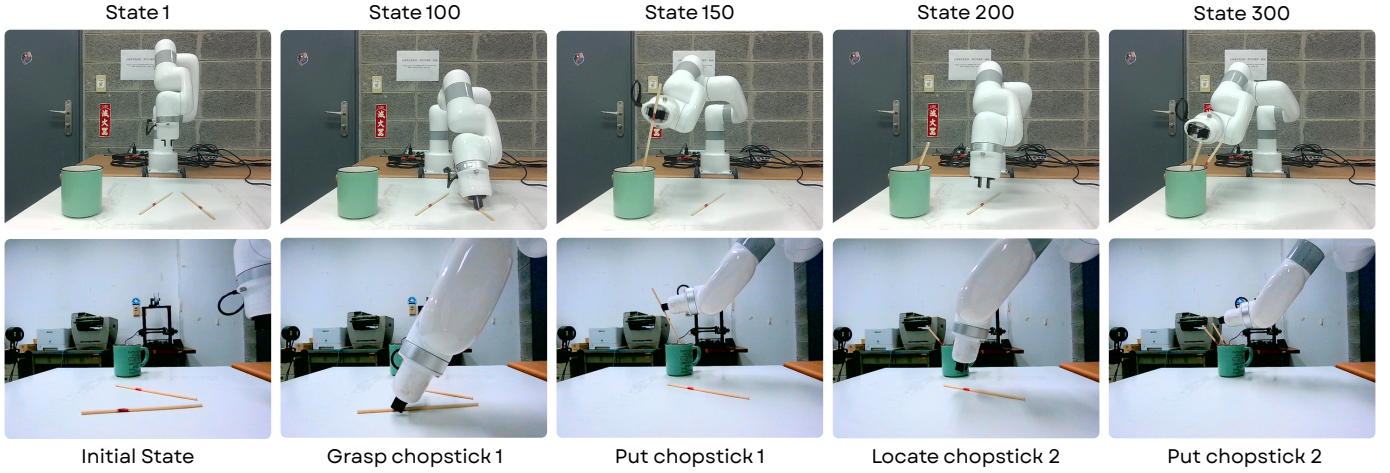


Fig. 7: **Chopsticks task demonstrations.** The steps in this illustration correspond to the robot states from state 1 to state 300.

TABLE III: Success rate (%) for the Chopsticks task is evaluated in three stages.

	1 st Chopstick			2 nd Chopstick		
	Locate	Lift	Put	Locate	Lift	Put
Proficient Human	96	96	94	94	94	92
BC-ConvMLP	0	0	0	0	0	0
ACT	54	50	44	44	40	34
MEAT*	88	74	66	56	56	48

task. MEAT achieves the highest success rates in all sub-tasks, with 88% success in locating the first chopstick, 74% in lifting, and 66% in placing it into the cup. The trend continues for the second chopstick, with MEAT achieving 56%, 56%, and 48% respectively, demonstrating strong consistency. In contrast, ACT shows a noticeable drop in performance, especially in the second chopstick phase, and BC-ConvMLP fails entirely in all stages. These results highlight the robustness and generalization capabilities of MEAT in sequential multi-step manipulation tasks under real-world constraints.

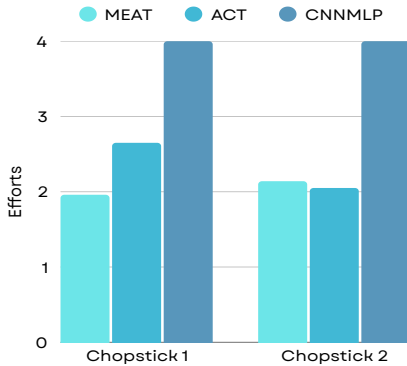


Fig. 8: **Chopsticks task average efforts until success or fail.** If the policy can not succeed after a maximum of 4 efforts, it will be marked as FAIL.

Retrying efforts. As explained, when the policy fails to locate the gripper over a chopstick or fails to grasp it, it automatically attempts to retry the action. Due to time constraints, we allow a maximum of four retries, as failure beyond this point typically results in continued failure for the remainder of the trial. As shown in **Figure 8**, we report the average number of retries per model. Fewer retries indicate more efficient and successful policy behavior. For Chopstick 1, MEAT averages 1.96 retries, compared to ACT’s 2.65. For Chopstick 2, while ACT appears to require slightly fewer retries, this count only applies if Chopstick 1 was successfully completed. Referring to **Table III**, we note that fewer ACT trials reached Chopstick 2, which influences the average and slightly inflates the apparent efficiency.

C. Task 2: ‘Plug Insertion’



Fig. 9: **Plug Insertion task.** This task involves: (1) picking up a power adapter, (2) bringing it to a plug and inserting it completely.

The second experiment, inspired by [30], is called ‘Plug Insertion’. As shown in the **Figure 9**, a plug is placed in front

TABLE IV: Success rate (%) for the 3 real-world tasks: ‘Plug Insertion’, ‘Stack 4’, and ‘Sort Pens & Chopsticks’.

Task	Plug Insertion		Stack 4			Sorting	
	Pick Up	Insertion	Stack 2	Stack 3	Stack 4	Pen	Chopsticks
Human	98	90	96	94	94	98	100
ConvMLP	0	0	0	0	0	0	0
ACT	94	46	22	10	2	69	59
MEAT	96	62	26	14	4	84	82

of a socket, and the robot must pick up the plug and insert it into the socket. This task is more contact-rich and requires greater precision. We trained the ACT and MEAT models for 25,000 steps, which took approximately six hours using 50 demonstrations. All demonstration trajectories were padded to 400 steps.

Result Analysis. Table IV’s first column presents the success rates for the plug insertion task. Both MEAT and ACT demonstrate high performance in the Pick up stage, achieving success rates of 96% and 94%, respectively—close to the human benchmark of 98%. The key difference lies in the Insertion stage: MEAT achieves a 62% success rate, notably higher than ACT’s 46%, highlighting MEAT’s superior efficiency. During experiments, we observed that ACT exhibited specific failure modes. For example, in the first training seed, the policy could guide the robot arm to the plug but failed to close the gripper. In the second seed, the policy navigated near the socket but took an excessively long time to lower the arm for insertion. These results suggest that, despite its competence in grasping, ACT suffers from stability issues during insertion.

D. Task 3: ‘Stack 4’

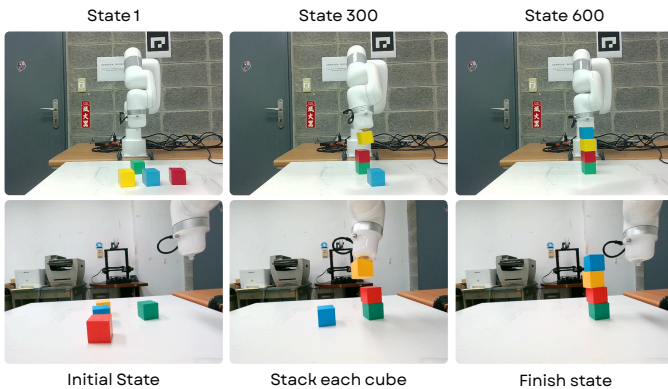


Fig. 10: Task Stack 4 demonstrations.

The third experiment is Stack 4. In general, this task involves stacking cubes together, a popular benchmark in robotic arm research due to its relevance to manipulation and control challenges. The goals for such tasks often vary depending on the setup and objectives. For instance, [33] focuses on stacking cubes of different shapes, while [12] builds a stone tower by stacking irregularly shaped stones. In our experiment, the initial setup include four cubes: Red, Yellow,

Blue, and Green, are placed on the table, with the Green cube positioned near the robot arm’s center. The objective is to stack the Red, Yellow, and Blue cubes on top of each other respectively, forming a stable tower without collapse. In this task, the robot uses a vacuum end-effector (Figure 10). Compared to the Chopsticks task, Stack 4 is significantly more challenging. The robot must accurately locate and grasp each cube and place it precisely on top of the stack, all within the limited perspective of 2 camera views. For training, we collected 65 demonstrations from proficient human operators and trained the model for 30,000 epochs, taking approximately 7 hours. Each trajectory spans 600 timesteps.

Result Analysis. The ‘‘Stack 4’’ task proves to be particularly challenging, shown in the second column of Table IV. Although MEAT still outperformed ACT, it was only able to fully complete approximately 10% of the tasks in both cases. Without human intervention, a notable highlight is MEAT’s ability to stack two blocks with a success rate of about 46%, compared to ACT’s 35%, and to stack three blocks at 34%, outperforming ACT’s 18%. We observed that the primary difficulty stemmed from the robot arm’s ability to accurately locate each cube for grasping. In contrast, placing the cubes after picking them up was relatively straightforward. Hence, we also carried out an experiment to test if humans facilitate the robot arm to locate the cube during policy inference would improve the performance. The results, as can be seen, have improved 20-30% at each period.

Limitation. However, the limitation of hardness to locate may be attributed to the performance of the image encoder or possibly the restricted camera setup; we used only two angles. To address this, our future work will involve enhancements to the algorithm as well as an increase in the number of camera perspectives.

E. Task 4: Sorting Pen & Chopsticks

We want to evaluate whether the policy can reliably distinguish between different objects. While this ability largely depends on the vision backbone of the policy, it is still an important factor to observe. In this task, two ink pens and two chopsticks will be placed in front of the robot (Figure 12). The robot’s objective is to pick up the two chopsticks and place them in the white box on the right, and to pick up the two pens and place them in the black pen holder on the left. As discussed in the previous section, the model’s main weakness lies in accurately localizing objects. To address this, our implementation provides additional support to help the

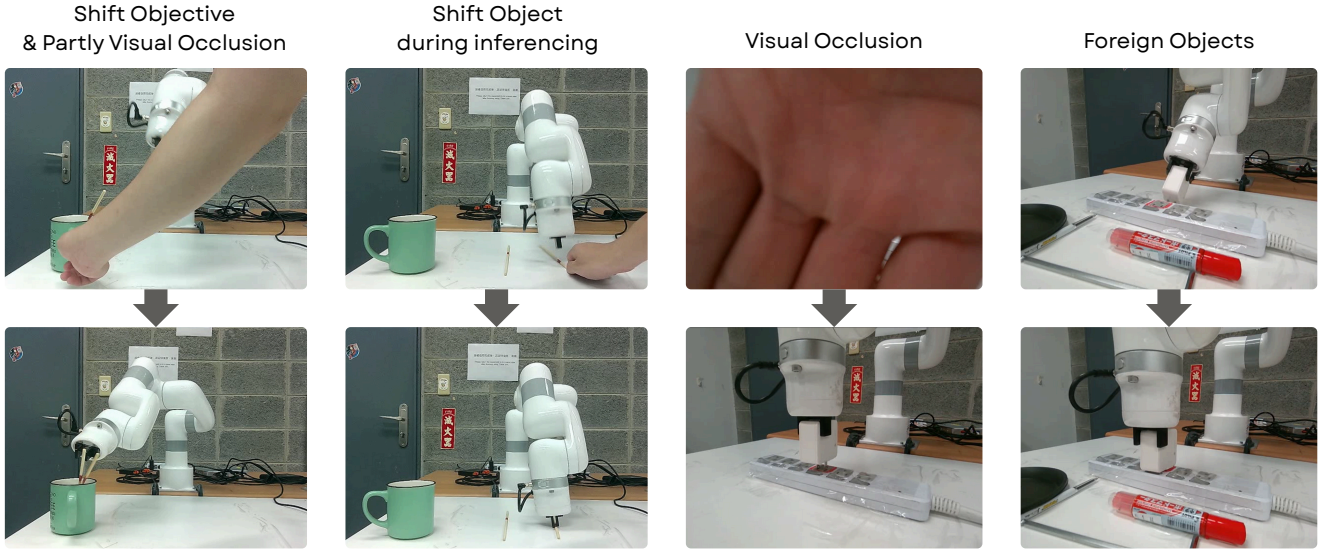


Fig. 11: Robustness Test for MEAT.

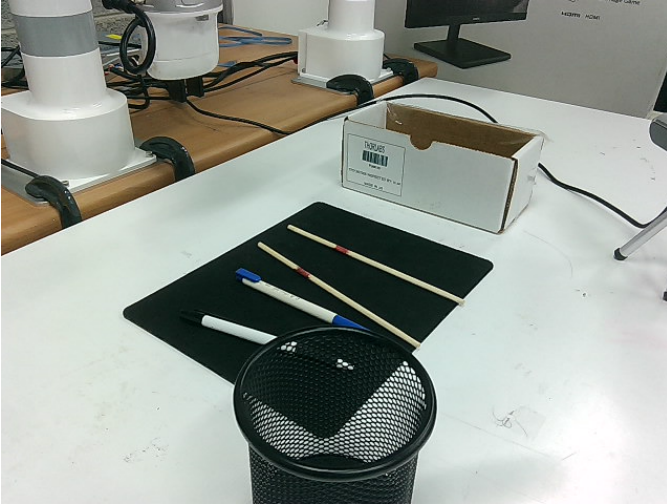


Fig. 12: **Sorting task demonstration.** The robot arm grasps each chopstick or pen, then places the chopsticks into the white box and the pens into the black pen holder.

model localize the objects more precisely. Therefore, the focus of this experiment is not on localization performance but rather on testing whether the policy can correctly distinguish the objects and place them in the appropriate locations.

Result Analysis For the Sorting task (pens and chopsticks), MEAT performs the best with 84% on pens and 82% on chopsticks, showing strong generalization and robustness in real-world object sorting. In contrast, ACT achieves only 69% on pens and 59% on chopsticks, indicating clear difficulties in reliably distinguishing and placing objects. Meanwhile, ConvMLP completely fails, with a 0% success rate in both categories, suggesting that simple architectures without temporal modeling or richer representations are insufficient for this task. Overall, the results highlight that while MEAT

narrows the gap toward human-level sorting, there is still room for improvement, especially in achieving consistent accuracy comparable to human performance. It is interesting to note that when the robot about to drop the chopstick or pen in wrong side, it will replanning to put it in other side, however we would see some confusion during the planning, the problem is that MEAT show less confusion while replanning while ACT is more confusion, and some time it drop the object right at the middle without choosing any box to put it in.

F. Robustness against perturbation

MEAT’s robustness against visual and physical perturbations was evaluated in the experiments shown in Figure 11, where we introduced disturbances such as partially covering the camera and shifting the object, moving the objects during inference, fully covering one camera view for three seconds, and throwing foreign objects during policy execution. The MEAT policy promptly adapted by adjusting the robot arm’s trajectory to compensate for these perturbations.

VI. ABLATIONS

In this section, we perform three ablations to analyze auxiliary loss, backbone depth, and normalization strategy in our setup.

A. Impact of the Auxiliary Loss Term

In this ablation study, we remove the auxiliary loss for expert load balancing to test its effect on performance. The results show only a small change: accuracy in the Insertion Scripted task drops by about 2%, while the Transfer Scripted task drops by around 1%. This indicates that the auxiliary loss does provide a positive effect, but the difference is relatively minor in this setup.

The limited impact can be explained by the fact that this experiment uses a small model, where the benefit of expert load balancing is less pronounced. In smaller models, the

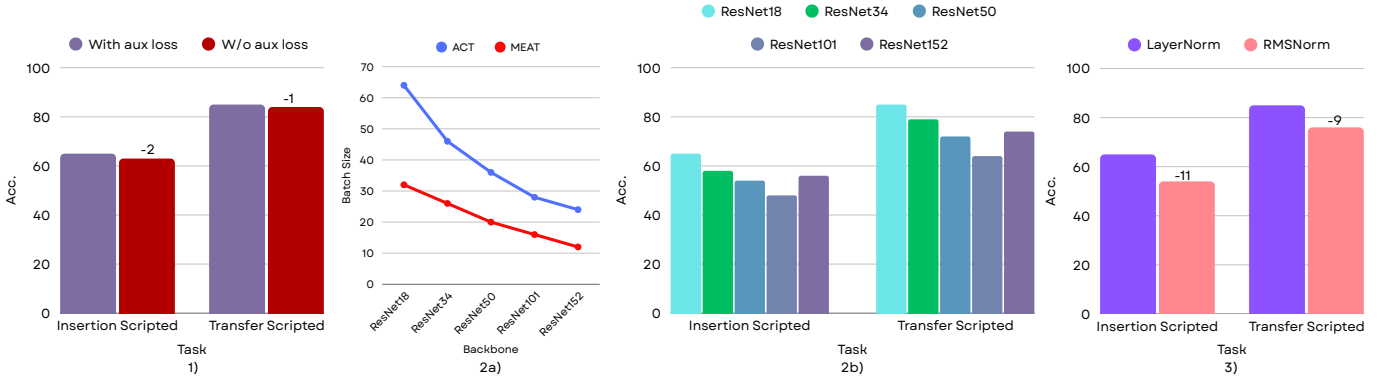


Fig. 13: **Ablation study results.** 1) Effect of auxiliary loss on accuracy for 2 tasks. 2) Impact of different ResNet backbones on batch size and accuracy. 3) Comparison of LayerNorm and RMSNorm on both tasks.

number of experts and their capacity are already constrained, so the imbalance between experts is not as severe. As a result, removing the auxiliary loss does not significantly harm accuracy, though it still offers slight improvements, particularly in tasks that demand higher precision.

B. Backbone Sensitivity study

We conduct an ablation study by replacing the original ResNet18 backbone with deeper ResNet variants (ResNet34, ResNet50, ResNet101, ResNet152) from [15] to assess their impact on the Insertion Scripted and Transfer Scripted tasks. ResNet, with its residual connections, enables training of deep convolutional networks by mitigating vanishing gradients. While deeper models can capture more complex features, they also increase memory usage and computational cost, which may limit batch size and, in some cases, lead to diminishing returns in performance.

Figure 13.2 investigates the impact of different ResNet backbones on both batch size efficiency and task accuracy. The original implementation uses ResNet18, a relatively shallow architecture that is computationally lighter and more memory-efficient. As shown in Figure 13.2a, where we capture the maximum batch size of each resnet on our setup, ResNet18 supports the largest batch size, while deeper networks such as ResNet101 and ResNet152 significantly reduce the feasible batch size due to higher parameter counts and memory requirements. Since all models were trained for the same number of epochs, a smaller batch size effectively means fewer overall training steps, which may limit convergence for deeper backbones.

From the accuracy perspective (Figure 13.2b), deeper ResNet variants do not consistently improve performance across the Insertion Scripted and Transfer Scripted tasks. In fact, ResNet18 performs competitively, achieving the highest or near-highest accuracy. This suggests that deeper models may require longer training schedules to fully utilize their capacity, while under the current setting, their reduced effective batch size and increased complexity can hinder optimization.

Overall, these results show that deeper is not always better for visual backbones in robotic manipulation. ResNet18 offers

a favorable trade-off between efficiency and performance, highlighting the importance of aligning model capacity with task complexity and available training budget.

C. Replacing LayerNorm with RMSNorm

We compared RMSNorm [1] and LayerNorm [42] on two small-scale experiments from ALOHA dataset (*Insertion Scripted* and *Transfer Cube Scripted*), each averaged over 3 seeds.

Formally, LayerNorm standardizes activations by both mean and variance,

$$\hat{a}_i = \frac{a_i - \mu(a)}{\sigma(a)}, \quad y_i = \gamma_i \hat{a}_i + \beta_i, \quad (8)$$

whereas RMSNorm discards the mean and rescales only by the root mean square,

$$\tilde{a}_i = \frac{a_i}{\sqrt{\frac{1}{d} \sum_{i=1}^d a_i^2 + \epsilon}}, \quad y_i = \gamma_i \tilde{a}_i + \beta_i, \quad (9)$$

As shown in the Figure 13.3, replacing LayerNorm with RMSNorm consistently reduces final performance: accuracy dropped by 11% on *Insertion Scripted* and 9% on *Transfer Scripted*. While RMSNorm is computationally cheaper by removing mean-centering and normalizes only by the root mean square, our results suggest that the re-centering provided by LayerNorm plays a stabilizing role in low-data or small-model regimes. Prior works [42] and [25] show that RMSNorm yields 5%~64% speedups with generally comparable quality, and in large-scale settings, this trade-off can be negligible. However, our findings highlight that in smaller setups, mean-centering contributes noticeably to optimization stability and generalization, making LayerNorm the more robust choice despite its higher cost.

VII. CONCLUSION & LIMITATIONS

In this paper, we propose MEAT, where the MoE architecture is integrated into a Transformer-based policy to control a robotic arm. Through 6 simulation experiments and 4 real experiments across various tasks, our model consistently

outperformed the baseline Transformer by margins ranging from 5% to 30%, demonstrating improved learning efficiency and control precision. Furthermore, we tested the proposed approach with auxiliary-loss-free MoE routing mechanism as an additional experiment, but it did not yield prominent results, highlighting the need for further investigation into more effective balancing strategies and loss function design. While our approach achieves state-of-the-art performance, we observed that training stability can be sensitive to the choice of random seed, with some seeds producing significantly better outcomes than others, a phenomenon also reported in prior work. Finally, this study focuses primarily on detailed architectural improvements; future research will explore integrating the model into more general frameworks, such as those involving text goal actions or vision-language models.

REFERENCES

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer normalization”. In: *Advances in NIPS Deep Learning Symposium*. 2016.
- [2] Kevin Black et al. “ π_0 : A Vision-Language-Action Flow Model for General Robot Control”. In: *CoRR* abs/2410.24164 (2024).
- [3] Anthony Brohan et al. “RT-1: Robotics Transformer for Real-World Control at Scale”. In: *Proceedings of Robotics: Science and Systems*. Daegu, Republic of Korea, July 2023. DOI: 10.15607/RSS.2023.XIX.025.
- [4] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. “End-to-End Object Detection with Transformers”. In: *Computer Vision – ECCV 2020*. Ed. by Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm. Cham: Springer International Publishing, 2020, pp. 213–229. ISBN: 978-3-030-58452-8.
- [5] Cheng Chi et al. “Diffusion policy: Visuomotor policy learning via action diffusion”. In: *The International Journal of Robotics Research* 0.0 (0), p. 02783649241273668. DOI: 10.1177/02783649241273668.
- [6] Sudeep Dasari, Oier Mees, Sebastian Zhao, Mohan Kumar Srirama, and Sergey Levine. “The Ingredients for Robotic Diffusion Transformers”. In: *International Conference on Robotics and Automation (ICRA)*. 2024.
- [7] DeepSeek-AI et al. *DeepSeek-V3 Technical Report*. 2025. arXiv: 2412.19437 [cs.CL].
- [8] Nan Du et al. “GLaM: Efficient Scaling of Language Models with Mixture-of-Experts”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 5356–5368.
- [9] William Fedus, Barret Zoph, and Noam Shazeer. “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity”. In: *J. Mach. Learn. Res.* 23.1 (Jan. 2022). ISSN: 1532-4435.
- [10] Pete Florence et al. “Implicit Behavioral Cloning”. In: *Proceedings of the 5th Conference on Robot Learning*. Ed. by Aleksandra Faust, David Hsu, and Gerhard Neumann. Vol. 164. Proceedings of Machine Learning Research. PMLR, Aug. 2022, pp. 158–168.
- [11] Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. “Mobile ALOHA: Learning Bimanual Mobile Manipulation using Low-Cost Whole-Body Teleoperation”. In: *8th Annual Conference on Robot Learning*. 2024.
- [12] Fadri Furrer et al. “Autonomous robotic stone stacking with online next best object target pose planning”. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 2017, pp. 2350–2356. DOI: 10.1109/ICRA.2017.7989272.
- [13] Dibya Ghosh et al. “Octo: An Open-Source Generalist Robot Policy”. In: *Proceedings of Robotics: Science and Systems*. Delft, Netherlands, July 2024. DOI: 10.15607/RSS.2024.XX.090.
- [14] Hussein Hazimeh et al. “DSelect-k: Differentiable Selection in the Mixture of Experts with Applications to Multi-Task Learning”. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan. Vol. 34. Curran Associates, Inc., 2021, pp. 29335–29347.
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising diffusion probabilistic models”. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS ’20. Vancouver, BC, Canada: Curran Associates Inc., 2020. ISBN: 9781713829546.
- [17] Albert Q. Jiang et al. *Mixtral of Experts*. 2024. arXiv: 2401.04088 [cs.LG].
- [18] Alexander Khazatsky et al. “DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset”. In: *RSS 2024 Workshop: Data Generation for Robotics*. 2024.
- [19] Lucy Lai, Ann Z.X. Huang, and Samuel J. Gershman. “Action chunking as conditional policy compression”. In: *Cognition* 264 (2025), p. 106201. ISSN: 0010-0277. DOI: <https://doi.org/10.1016/j.cognition.2025.106201>.
- [20] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. “BASE Layers: Simplifying Training of Large, Sparse Models”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 6265–6274.
- [21] Senmao Li et al. “Faster Diffusion: Rethinking the Role of the Encoder for Diffusion Model Inference”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024.
- [22] Ajay Mandlekar et al. “MimicGen: A Data Generation System for Scalable Robot Learning using Hu-

- man Demonstrations”. In: *Towards Generalist Robots: Learning Paradigms for Scalable Skill Acquisition @ CoRL2023*. 2023.
- [23] Ajay Mandlekar et al. “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation”. In: *5th Annual Conference on Robot Learning*. 2021.
- [24] Ashvin Nair et al. “Combining self-supervised learning and imitation for vision-based rope manipulation”. In: *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2017, pp. 2146–2153.
- [25] Sharan Narang et al. “Do Transformer Modifications Transfer Across Implementations and Applications?” In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Ed. by Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 5758–5773.
- [26] Abby O’Neill et al. “Open X-Embodiment: Robotic Learning Datasets and RT-X Models : Open X-Embodiment Collaboration”. In: *ICRA*. 2024, pp. 6892–6903.
- [27] Jyothish Pari, Nur Muhammad (Mahi) Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. “The Surprising Effectiveness of Representation Learning for Visual Imitation”. In: *Proceedings of Robotics: Science and Systems*. New York City, NY, USA, June 2022. DOI: 10.15607/RSS.2022.XVIII.010.
- [28] Jyothish Pari, Nur Muhammad (Mahi) Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. “The Surprising Effectiveness of Representation Learning for Visual Imitation”. In: *Proceedings of Robotics: Science and Systems*. New York City, NY, USA, June 2022. DOI: 10.15607/RSS.2022.XVIII.010.
- [29] Dean A Pomerleau. “Alvin: An autonomous land vehicle in a neural network”. In: *Advances in neural information processing systems* 1 (1988).
- [30] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. “Consistency Policy: Accelerated Visuomotor Policies via Consistency Distillation”. In: *Robotics: Science and Systems*. 2024.
- [31] Vignesh Prasad, Alap Kshirsagar, Dorothea Koert, Ruth Stock-Homburg, Jan Peters, and Georgia Chalvatzaki. “MoVEInt: Mixture of Variational Experts for Learning Human–Robot Interactions From Demonstrations”. In: *IEEE Robotics and Automation Letters* 9.7 (2024), pp. 6043–6050. DOI: 10.1109/LRA.2024.3396074.
- [32] “Q-Transformer: Scalable Offline Reinforcement Learning via Autoregressive Q-Functions”. In: *7th Annual Conference on Robot Learning*. 2023.
- [33] Scott Reed et al. “A Generalist Agent”. In: *Transactions on Machine Learning Research* (2022). Featured Certification, Outstanding Certification. ISSN: 2835-8856. URL: <https://openreview.net/forum?id=likK0kHjvj>.
- [34] Moritz Reuss, Jyothish Pari, Pulkit Agrawal, and Rudolf Lioutikov. “Efficient Diffusion Transformer Policies with Mixture of Expert Denoisers for Multitask Learning”. In: *The Thirteenth International Conference on Learning Representations*. 2025.
- [35] Moritz Reuss, Ömer Erdiñç Yağmurlu, Fabian Wenzel, and Rudolf Lioutikov. “Multimodal Diffusion Transformer: Learning Versatile Behavior from Multimodal Goals”. In: *Proceedings of Robotics: Science and Systems*. Delft, Netherlands, July 2024. DOI: 10.15607/RSS.2024.XX.121.
- [36] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. “Hash layers for large sparse models”. In: *Proceedings of the 35th International Conference on Neural Information Processing Systems*. NIPS ’21. Red Hook, NY, USA: Curran Associates Inc., 2021. ISBN: 9781713845393.
- [37] Nur Muhammad (Mahi) Shafiullah, Ziehen Jeff Cui, Ariuntuya Altanzaya, and Lerrel Pinto. “Behavior transformers: cloning k modes with one stone”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. New Orleans, LA, USA: Curran Associates Inc., 2022. ISBN: 9781713871088.
- [38] Noam Shazeer et al. “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer”. In: *International Conference on Learning Representations*. 2017.
- [39] Ashish Vaswani et al. “Attention is all you need”. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS’17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010. ISBN: 9781510860964.
- [40] Eric Wallace, Olivia Watkins, Miles Wang, Kai Chen, and Chris Koch. *Estimating Worst-Case Frontier Risks of Open-Weight LLMs*. 2025. arXiv: 2508.03153. URL: <https://arxiv.org/abs/2508.03153>.
- [41] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. *Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts*. 2024. arXiv: 2408.15664 [cs.LG].
- [42] Biao Zhang and Rico Sennrich. “Root mean square layer normalization”. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [43] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware”. In: *Proceedings of Robotics: Science and Systems*. Daegu, Republic of Korea, July 2023. DOI: 10.15607/RSS.2023.XIX.016.
- [44] Yanqi Zhou et al. “Mixture-of-experts with expert choice routing”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. New Orleans, LA, USA: Curran Associates Inc., 2022. ISBN: 9781713871088.

APPENDIX

A. Teleoperation Method

Algorithm 1 RecordData(arm_1, arm_2, cams)

```

1:  $t_0 \leftarrow$  current time
2: while time elapsed < duration do
3:    $a \leftarrow$  servo_angle(arm_1) + eef
4:    $q \leftarrow$  joint_pos(arm_2),  $\theta \leftarrow$  servo_angle(arm_2)
5:    $I \leftarrow$  capture images from all cameras
6:   Store  $a, q, \theta, I$  in HDF5
7:   if key 'q' then
8:     break
9:   end if
10: end while

```

This system implements a multithreaded teleoperation and data recording pipeline for a dual-arm robot setup. As described in **Algorithm 4**, three concurrent threads are launched to handle key subsystems: end-effector (EEF) control, arm synchronization, and sensor-data logging. Each component operates asynchronously and communicates via shared state (eef_state) and a global termination flag (stop_flag).

The EEFCtrl thread (see **Algorithm 2**) monitors keyboard input to control the slave robot's end-effector. Pressing '1' opens the end-effector, and '0' closes it. The binary state is stored in a shared list eef_state, making it accessible to both synchronization and logging threads.

Algorithm 2 EEFCtrl(arm)

```

1: while not stop_flag do
2:   if key '1' then
3:     open_eef(arm), eef_state  $\leftarrow$  1
4:   end if
5:   if key '0' then
6:     close_eef(arm), eef_state  $\leftarrow$  0
7:   end if
8: end while

```

Meanwhile, the SyncArms thread (**Algorithm 3**) continuously reads the joint angles of the master arm and appends the eef_state value. These combined control signals are sent to the slave arm using velocity commands. The loop executes at 100Hz (10ms intervals) to maintain smooth and precise motion replication.

Algorithm 3 SyncArms(master, slave)

```

1: while not stop_flag do
2:    $\theta \leftarrow$  servo_angle(master) + eef
3:   set_angle(slave,  $\theta[1:6]$ )
4:   sleep(0.01)
5: end while

```

The RecordData function (**Algorithm 1**) logs the master arm's action (servo angles and EEF state), the slave arm's joint states, and image frames from connected USB cameras. The

data is serialized and stored in an HDF5 file, structured into action, qpos, qangl, and per-camera image datasets. Recording continues until the defined time limit or user termination.

Algorithm 4 Multithreaded Teleoperation and Data Recording

```

1: Init: stop_flag  $\leftarrow$  false, eef_state  $\leftarrow$  1
2: Start thread: EEFCtrl(arm_2)
3: Start thread: SyncArms(arm_1, arm_2)
4: Start thread: RecordData(arm_1, arm_2, cams)
5: Wait for RecordData thread to finish
6: Signal stop_flag
7: Join all threads
8: Release cameras, close HDF5, reset arms

```

Finally, once data recording is complete, all threads are terminated cleanly using stop_flag, resources are released, and the arms are returned to their home configuration. This design achieves real-time synchronized teleoperation with integrated multimodal data collection, suitable for robot learning applications such as behavior cloning or imitation learning.

B. Hardware Setup Specific

TABLE V: Robot arm specification

Specification	LITE 6	XARM 6
Payload	600g	5kg
Reach	440mm	700mm
Degrees of Freedom	6	6
Repeatability	± 0.5 mm	± 0.1 mm
Maximum Speed	500mm/s	1m/s
Weight (arm only)	7.2kg	12.2kg
Gripper stroke range	0-16mm	-
Reverse stroker range	20-38mm	-

C. Training Loss

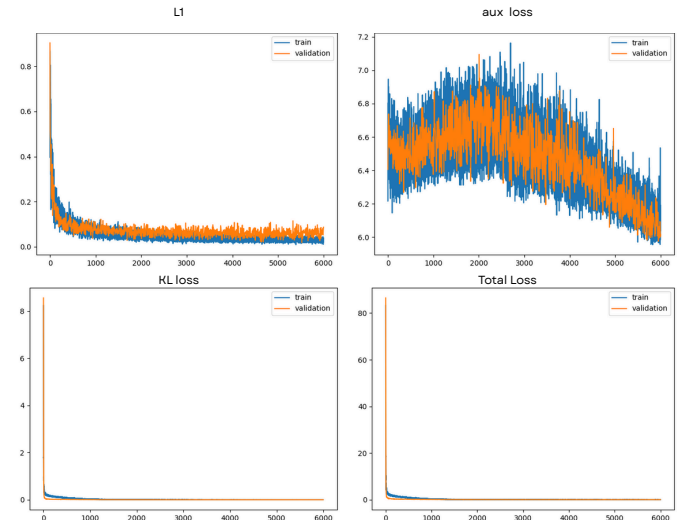


Fig. 14: A sample of training loss.

D. Hyperparameters config

We carefully tune the baselines and include the hyperparameters used in Tables VI, VII, VIII.

TABLE VI: Task specific configs

Task	Common				Transformer models							
					Common		ACT			MEAT		
	Is_real	Views	Chunk size	EpLen	#Episode	ImageRes	BS	LR	Epochs	BS	LR	Epochs
Transfer Cube	F	Top View	100	400	50	480x640	64	5e-5	2500	32	3e-5	2500
Cube Insertion	F	Top View	100	500	50	480x640	64	5e-5	2500	32	3e-5	2500
Tool hang	F	Agent View	20	700	150	480x640	64	5e-5	4000	32	3e-5	4000
Tool transport	F	Agent View	100	600	150	480x640	64	5e-5	4000	32	3e-5	4000
Chopstick	T	Side View, Front View	20	400	49	480x640	48	4e-5	15000	24	2e-5	15000
Stack 4	T	Side View, Front View	20	700	66	480x640	36	3e-5	30000	22	2e-5	30000

TABLE VII: Task specific configs (cont.)

Task	BC-RNN					Diffusion				
	#Episode	ImageRes	BS	LR	epochs	#Episode	ImageRes	BS	LR	epochs
Tool hang	150	480x640	256	1e-4	1000	150	480x640	128	1e-4	2000
Tool transport	150	480x640	256	1e-4	1000	200	240x320	172	1e-4	2000

TABLE VIII: Models base common configs

Transformer models		Diffusion & BC-RNN	
Hyper params	Config	Hyper params	Config
#Encoder layers	4	#Layers	8
#Decoder layers	7	#V-params	22
Feedforward dimension	3200	#D-params	9
Hidden dimension	512	Hidden dimension	255
Ctrl	Pos	Ctrl	Pos
KL weight	10	Weight decay	1e-3
# Experts	4	Steps per Epoch	500
Dropout	0.1	Dropout	0.3
Heads	8	Warmup steps	500
Seed	0,42,100	Seeds	0,42,100
Pretrained Encoder	Resnet18	Pretrained Encoder	Resnet18
Auxiliary weight	1e-3		